

CogniSeek AI: The Complete Project Guide

A personal study guide: how every part works, how the parts connect, and why each was built that way. Written so you can explain the project to anyone, from an HR screener to a senior engineer.

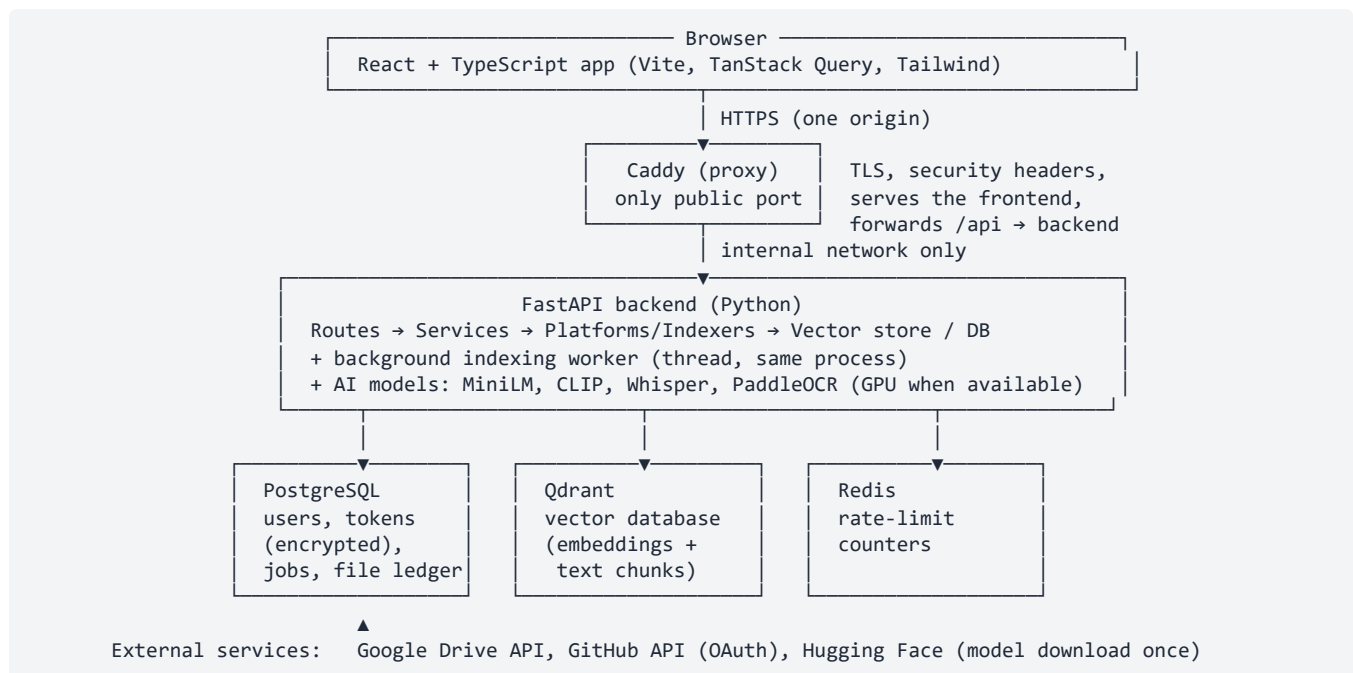
Part 1: The 30-second explanation (for HR / non-technical people)

"CogniSeek is a search engine for **your own files**. People keep files in many places: their laptop, Google Drive, GitHub. They forget names and locations. CogniSeek connects to all of them, reads and *understands* the files (documents, code, images, audio, video) using AI, and lets you search in plain English, like '*find my Java notes*' or '*video with polar bear cubs*'. It finds files by **meaning**, not just by exact filename. It is multi-user and secure: each person only ever sees their own files, and their cloud passwords/tokens are encrypted."

What makes it interesting technically (one line each):

1. **Multimodal AI search:** text, code, images, audio and video all in one search box.
 2. **Hybrid retrieval:** combines AI meaning-search with classic filename/keyword search, calibrated so junk results are filtered out.
 3. **Real integrations:** Google Drive and GitHub through proper OAuth, with incremental sync (only changed files are re-processed).
 4. **Security-first design:** per-user data isolation, encrypted tokens, hardened API, HTTPS deployment.
 5. **Engineered like a product:** 450+ backend tests, 70+ frontend tests, an end-to-end test, an evaluation harness, Docker deployment, and measured performance.
-

Part 2: The big picture (architecture)



Each component and why it exists:

Component	Job	Why this choice
React frontend	The UI: search, platforms, indexing center, onboarding	Typed API client generated from the backend's OpenAPI schema, so frontend and backend can't drift apart
Caddy	Reverse proxy: HTTPS, headers, a single entry point	Automatic TLS certificates; the frontend and API share one origin, so no CORS is needed in production
FastAPI	REST API and business logic	Fast, typed with Pydantic validation, auto-generated OpenAPI docs
Indexing worker	Background thread that processes indexing jobs	Indexing is slow; it must not block API requests
PostgreSQL	Relational data: users, connections, jobs, the per-file ledger	ACID transactions, row locking for the job queue, trigram search on filenames
Qdrant	Vector database: embeddings plus text chunks	Fast nearest-neighbour search with payload filters (needed for per-user isolation)
Redis	Rate-limit counters	Shared across restarts and workers
AI models	Turn content into vectors or text	See Part 5

Part 3: How a request flows through the backend (layers)

HTTP request

- Middleware: request-id, security headers, CORS (allowed origins only), rate limiting, error handling
- Route (`app/routes/*.py`): validates input with Pydantic models, requires auth
- Dependency `get_current_user` (`app/auth/auth_dependency.py`): verifies the JWT, loads the user
- Service (`app/services/*.py`): business logic (`search_service`, `index_store`, `account_service...`)
- Platform / indexer / vectorstore / database
- Response model (`app/models/response_models.py`): typed JSON back

Important folders:

- `app/routes/` : API endpoints (auth, search, index, platforms, github, google_drive, files, open, upload, delete, dashboard, health).
 - `app/auth/` : password hashing, JWT creation/verification, the current-user dependency.
 - `app/core/` : cross-cutting concerns: config, crypto (token encryption), path safety, rate limiting, security headers, logging, request IDs, metrics.
 - `app/platforms/` : connectors for local, google_drive and github, plus shared sync, retry (`http.py`) and OAuth state.
 - `app/services/` : the indexing pipeline, `index_store` (the only writer to Qdrant and the ledger), search, account, dashboard.
 - `app/extractors/` : read text from PDF/DOCX/PPTX, OCR, video frames, audio.
 - `app/search/` : retrieval, ranking, calibration, query normalization.
 - `app/vectorstore/` : the Qdrant client, collection schema, and **the single query function** (isolation choke-point).
 - `app/scheduler/` : the job queue and worker.
 - `alembic/versions/` : 15 database migrations, the versioned history of the schema.
-

Part 4: Authentication (how login works, step by step)

4.1 Registration

1. The user sends name, email and password to `POST /auth/register`.
2. **Validation (Pydantic)**: a valid email format (lower-cased); a name of 1–100 characters; a password of at least 8 characters with at least one letter and one digit, at most 128 characters, and **at most 72 bytes** (bcrypt's limit; longer passwords would be silently truncated, so they are rejected).
3. The password is hashed with **bcrypt** (`app/auth/password.py`). bcrypt is deliberately slow and **salted**: the same password gives a different hash each time, so leaked hashes can't be cracked with precomputed tables. **The plain password is never stored.**
4. The user row is saved in PostgreSQL with `token_version = 0`.
5. **Rate limited**: 5 attempts per minute per IP (Redis-backed), which stops mass sign-ups and brute force.

4.2 Login

1. `POST /auth/login` with email and password (also rate-limited to 5/minute).
2. The server finds the user and runs `bcrypt.verify(password, hash)`. On failure it returns the **same generic message** whether the email exists or not, so attackers can't discover which emails are registered.
3. On success it creates a **JWT (JSON Web Token)** signed with **HS256** using the secret `JWT_SECRET_KEY` (from `.env`, at least 32 characters, required in production). The token contains: - `user_id`: who you are - `tv`: token version (for logout/revocation, see below) - `jti`: a unique token ID - `iat` / `exp`: issued-at and expiry (**60 minutes**)
4. The frontend stores the token and sends it on every request as `Authorization: Bearer <token>`.

4.3 Every protected request

`get_current_user` (FastAPI dependency):

1. Reads the Bearer token. If it's missing, the response is **401**.
2. Verifies the **signature** (was it signed with our secret?) and the **expiry**. A tampered or expired token gets **401**.
3. Loads the user **by** `user_id` (not email; IDs never change).
4. Compares the token's `tv` with the user's current `token_version`. If they differ, the token was revoked, so **401**.

Every route requires this, except register, login, `GET /`, the health/readiness checks (`/health`, `/ready`, `/search/health`) and the OAuth callbacks. (`/metrics` has no auth either, but it is only reachable on the internal Docker network; Caddy answers 404 for it.) An automated test (`tests/test_auth_required.py`) reads *all* routes from the app's OpenAPI schema and checks that each one returns 401 without a token, so a new route can never accidentally be left open.

4.4 Logout, password change, account deletion

- **Logout:** increments `token_version`. Every existing token for that user instantly fails the `tv` check. That's revocation without storing a blacklist. Logout also cancels the user's running indexing jobs.
- **Change password:** requires the current password, applies the same policy, and bumps `token_version`, so all other sessions are logged out.
- **Delete account:** requires the password. It revokes the Google/GitHub tokens at the providers, deletes all the user's vectors from Qdrant, all database rows, and uploaded files. The session dies immediately.
- **Export my data:** a JSON download of the profile, connections (names only, **never tokens**), folders, jobs and file list (no file contents).

4.5 Why JWT, and why it's stored in localStorage

- **JWT is stateless:** the server verifies the signature without a database lookup per request (except the cheap `user/tv` check).
- The weakness of localStorage is **XSS** (malicious script could read it). The mitigations are a **strict Content Security Policy** (scripts only from our own origin, no inline scripts), React's automatic escaping, no `dangerouslySetInnerHTML`, and a short 60-minute expiry.
- **Future improvement** (honest answer): move to an httpOnly cookie plus a CSRF token. It's documented in SECURITY.md.

Part 5: The AI models (what each one does)

Model	Used for	Output
all-MiniLM-L6-v2 (SentenceTransformers)	Text meaning: document chunks, code, transcripts, the search query	384-number vector
CLIP ViT-B/32 (OpenAI, via Hugging Face)	Images and video frames vs text; it can compare a picture with words	512-number vector
Whisper "base" (faster-whisper)	Speech → text for audio and video	Transcript text
PaddleOCR	Text inside images and scanned PDFs (ID cards, screenshots)	Text

Key ideas to explain:

- **Embedding:** an AI model turns content into a list of numbers (a vector) so that **similar meaning produces nearby vectors**. "Car" and "automobile" end up close together, even though the letters

differ.

- **Cosine similarity:** measures how close two vectors are (the angle between them). Qdrant finds the nearest vectors quickly.
- **CLIP is special:** it was trained on image–caption pairs, so a text query like "dog" lands near *pictures of dogs*, and we can search images by their content without any labels.
- **The model manager** (`app/ai/model_manager.py`) loads each model **once**, lazily, on the GPU if available (CUDA), with a **lock per model** so concurrent requests take turns instead of crashing the GPU. Nothing loads at import time; models preload at startup when `PRELOAD_MODELS` is true.

Part 6: Indexing (how files become searchable)

6.1 The job system

1. The user clicks **Index** (or finishes onboarding). `POST /index/` creates one **job row per platform** in PostgreSQL with status `queued`. - Duplicate protection: a database **partial unique index** allows only one queued or running job per user and platform (otherwise 409). - **Priority:** the platform chosen as priority gets a higher priority value, so it's processed first.
2. **One background worker thread** claims the next job with `SELECT ... ORDER BY priority DESC, created_at FOR UPDATE SKIP LOCKED`. - `FOR UPDATE` locks the row so nobody else takes it; `SKIP LOCKED` lets another worker skip it instead of waiting. This is the standard way to use Postgres as a reliable queue.
3. **Job states:** `queued` → `running` → `completed` | `completed_with_errors` | `failed` | `cancelled`.
4. **Reliability:** - Every file is processed in its own try/except. One bad file is recorded in `indexing_job_errors` (with paths and tokens sanitized) and the job continues. - The worker loop can't die; it catches its own errors. - **On server restart**, any job left `running` is marked `failed: Interrupted by server restart`, and orphaned temp folders are cleaned up. - **Cancel** stops after the current file(s) and also cancels the user's queued jobs.
5. **Why one worker?** All jobs share the same GPU models. Two jobs at once would compete for GPU memory without finishing sooner.

6.2 The pipeline for one file

```
list remote metadata → exclusion rules → needs_index? (ledger) → download to private temp dir  
→ extract (by type) → chunk → embed → upsert vectors to Qdrant (batched) → update ledger → delete temp file
```

By file type:

- **Documents** (PDF, DOCX, PPTX, TXT, MD, CSV, code): extract text. Scanned or image-heavy PDF pages get **OCR**: a page is OCR'd if it has fewer than 30 meaningful characters, if its text is mostly noise, or if images cover at least 40% of it (max 20 pages). Text is split into chunks, and each chunk becomes a MiniLM vector.
- **Images** (JPG, PNG, WEBP, AVIF): a CLIP image vector, plus OCR text.
- **Audio** (MP3, WAV, M4A, AAC, FLAC): a Whisper transcript (silence skipped with voice-activity detection), chunked, then MiniLM vectors.
- **Video** (MP4, AVI, MOV, MKV): the transcript as above, **plus one frame every 5 seconds**. Each frame gets a CLIP vector and a timestamp. Frames are extracted by **seeking** directly to each timestamp instead of decoding every frame (4.8–6.6× faster on the benchmark videos, pixel-identical). A per-video "null baseline" is computed (see Search).

6.3 Change detection (why re-indexing is fast)

The **ledger** (`indexed_files` table, one row per file per user) stores each file's **version**:

- **Local**: the file's modification time (`mtime`).
- **Google Drive**: `modifiedTime` (+ `md5Checksum`).
- **GitHub**: the **blob SHA** (a Git content hash; it changes exactly when the content changes).

Before downloading anything, the connector asks `needs_index(user, platform, file, version)`. If it's the same version, the file is skipped. **A second run on an unchanged Drive or GitHub downloads 0 files.** Files that produced no text, unsupported files and excluded files are also recorded, so they aren't retried forever.

6.4 Writing to the index safely

`index_store` (`app/services/index_store.py`) is the **only** module that writes or deletes vectors.

- **Deterministic point IDs**: `uuid5(user | platform | source_id | type | chunk | frame)`. Re-indexing a file **overwrites** the same points instead of creating duplicates.
- **Upsert-then-prune**: new chunks are written first (in batches of 256); only then are leftover old chunks deleted. A search running at the same moment never sees a file with zero vectors.
- **Limits**: at most 2,000,000 characters and 2,000 chunks per file (huge generated files are truncated, not crashing), and at most 200 MB per download. `*.log`, `*.lock`, `*.min.js`, `*.map` and similar junk files are excluded.

6.5 Deletion sync

- **Local**: after scanning, files no longer present under the user's folders (deleted files or removed folders) are deleted from the index.
- **Drive/GitHub**: after a **complete** listing, files missing from it are deleted. If the listing was partial (an error midway), deletion is **skipped**; otherwise a network glitch could wipe valid files.

6.6 Performance (Phase 7A.5, measured)

- Profiling showed OCR took 61% of the time while the GPU sat idle 86% of the time.
- Fixes: seek-based frame extraction, opt-in GPU OCR, batched PDF rendering, a parallel file pipeline (4 files at once, per-model GPU locks), batched CLIP frames, and **smallest-files-first** scheduling.
- Result: a 69-file benchmark went from **347 s** → **93 s** with GPU OCR (292 s on CPU), and half of the files became searchable after **31 s instead of 191 s**. Search quality was verified unchanged with the eval harness.
- The Indexing Center shows **"Where the time went"** per stage.

Part 7: The connectors (Local, Google Drive, GitHub)

7.1 Local storage

- The user registers folder paths (they're paths on the machine running the backend; in Docker, a mounted folder under `/data`).
- **Validation**: the folder must exist, must be a directory, must **not** be a drive root (`C:\`), and must be inside `ALLOWED_LOCAL_ROOTS` (by default the user's home directory). Paths are normalized, and

duplicates are blocked by a database unique constraint.

- **Opening a local result:** the browser downloads it through `GET /files/local`, which only serves a file if its **real resolved path** is inside one of *this user's* folders **and** the file is in this user's ledger; otherwise it returns 404.

7.2 Google Drive (OAuth 2.0 web flow with PKCE)

Connect, step by step:

1. The user clicks *Connect*. The backend generates a random **state** (`secrets.token_urlsafe(32)`) and a **PKCE code_verifier**, and stores both in the `oauth_states` table with the user id and a **10-minute expiry**.
2. The backend returns Google's authorization URL (with `code_challenge = SHA256(verifier)`), and the browser goes to Google.
3. The user picks an account and approves **read-only** access (`drive.readonly`, plus email/openid to show which account is connected).
4. Google redirects to `/platforms/google-drive/callback?code=...&state=...`.
5. The backend checks that the state **exists, is unexpired and unused**, marks it used, and takes the user **from the database row** (never from the URL).
6. It exchanges the code plus the **verifier** for tokens. A stolen code is useless without the verifier.
7. It checks the scopes Google **actually granted**. If Drive access is missing (an unticked checkbox), it saves nothing, revokes the token and shows a clear message.
8. It saves the tokens **encrypted** (Part 9) with the account email, then redirects to the app with a success toast.

Indexing: one Drive client per job; it lists all non-trashed files with pagination (1000 per page), skips folders, shortcuts and non-exportable types, and exports Google Docs/Slides/Sheets as DOCX/PPTX/CSV. Access tokens expire hourly; refresh is automatic and the new token is saved back encrypted. If Google says the grant is revoked (`invalid_grant`), the connection is marked disconnected and the user is asked to reconnect.

Open: opens the file's `webViewLink` in Google Drive (a new tab). **Disconnect:** revokes the token **at Google**, then deletes it locally (optionally also purging the indexed data).

7.3 GitHub (OAuth)

Same single-use state protection. Scopes are `repo` and `read:user`. **Indexing:**

- Lists repositories with **pagination** (100 per page, following the `Link` header); forks and archived repos are skipped by default.
 - For each repo, **one call** to the Git Trees API (`/git/trees/{default_branch}?recursive=1`) returns every file's path, size and **blob SHA**. (The older approach of one call per folder burns the rate limit.)
 - Downloads only new or changed blobs. Skips `node_modules`, `dist`, `venv`, lock files, minified files and Git LFS pointers.
 - File identity is `owner/repo:path`, so the same `README.md` in two repos stays separate.
 - **Rate limits:** a shared retry helper does exponential backoff with jitter, honours `Retry-After` and `X-RateLimit-Reset`, and pauses all parallel downloads on a 429. **Open:**
`https://github.com/{owner}/{repo}/blob/{default_branch}/{path}`.
-

Part 8: Search (how results are found and ranked)

8.1 Request

POST /search/ with `query` (1–500 characters after trimming), `search_type` (all/document/image/audio/video), `platform` (all/local/google_drive/github), `limit` / `offset`. Invalid values give a **422**. Rate-limited to 60/minute per user.

8.2 Retrieval (hybrid)

1. **Normalize** the query (lower-case, Unicode-normalize, strip punctuation), so "JAVA!!!" behaves like "java".
2. **Embed once per model**: MiniLM for text, CLIP for images and frames, with an LRU cache of 512 recent queries.
3. **Vector search in Qdrant**: queries for each relevant collection run **in parallel, always filtered by user_id** (and platform).
4. **Filename search in Postgres**: trigram similarity (`pg_trgm` + a GIN index) on file names, so exact or typo-ed filenames are found even when the content doesn't match.
5. Both candidate sets are merged per file.

8.3 Ranking and calibration (why junk is filtered out)

Each candidate gets three signals between 0 and 1: **semantic** (meaning), **filename match** and **content/keyword match** (text, OCR, transcript). They're combined with a "noisy-OR" ($1 - \prod(1 - w \cdot \text{signal})$), so each independent piece of evidence raises the score.

The hard part is calibration. Raw AI similarity scores are relative, so you can't threshold them directly:

- **Text**: the score is compared to how the query scores against **neutral prompts** (a "margin"). A margin of ≥ 0.35 counts as evidence for prose. Code files need ≥ 0.52 , because MiniLM barely separates code; a code file usually needs a name or content match.
- **Images**: CLIP gives almost everything about 0.2–0.3. An image is returned on visual evidence alone when its margin over neutral prompts is ≥ 0.030 (fitted from labelled data: it keeps about 70% of relevant images and lets through about 0.5% of irrelevant ones). Photos of documents (20 or more OCR words) need text evidence instead.
- **Videos**: some videos (for example a space documentary) "look similar" to everything. So each video's best frame is compared to **that same video's scores for 12 unrelated prompts** (a z-score). A score of ≥ 5.3 is a confident visual match.
- **Generic words** ("photo", "image", "file", "a", "the"...) never create a match on their own.
- **A result must have real evidence** (a strong semantic margin or a lexical hit). That's why nonsense like "xqzv wplk" returns nothing.

8.4 Two confidence tiers

- **Results**: confident matches, globally sorted, deduplicated per file, with `total` and pagination.
- **Possible visual matches**: a separate, clearly labelled list (max 3) of weaker visual matches (video z between 3.0 and 5.3, image margin between 0.015 and 0.030), never mixed into the results. Example: "river" shows the Forest Bathing video as *Low confidence · frame at 0:20*.

8.5 Each result contains

The file, platform, type, display path, size, modified date, score, **why it matched** (filename, content, visual, transcript), and a **highlighted snippet**. Videos also get the frame timestamp.

8.6 How quality is measured (the evaluation harness)

- `tests/eval/queries.yaml`: about 80 real queries with known correct answers, in subsets: documents/text, code, visual images, visual video, and **negative queries** (which should return nothing).
- `scripts/eval_search.py` computes **Precision@5** (how many of the top 5 are relevant), **Recall@10**, **MRR** (how high the first correct answer ranks), false positives and latency.
- **Every ranking change is accepted only if no subset regresses**. Example numbers: non-visual P@5 and MRR are about 0.935, with 0 negative false positives; visual images about 0.72; visual video 0.50 (up from 0).
- **Search latency**: p50 about 0.2–0.3 s warm over the eval queries (0.18–0.27 s in the latest runs), down from 1.5–2.3 s measured in the original audit.

Part 9: SECURITY: how user data is protected (the big question)

Think in **layers** ("defence in depth"). If one layer fails, others still protect the data.

Layer 1: Identity (who are you?)

- bcrypt-hashed, salted passwords; plain passwords are never stored or logged.
- Signed JWTs with expiry, server-side revocation (`token_version`), and the user looked up by an immutable ID.
- **Rate limiting** on login/register (5/minute), search (60/minute) and OAuth endpoints (10/minute), stored in Redis.

Layer 2: Authorization and isolation (what may you see?)

This answers "*can user A see user B's files?*". **No**, and here's why:

1. **Every piece of data is tagged with its owner**. Every Qdrant vector carries `user_id` in its payload; every Postgres row has a `user_id` column.
2. **The user_id comes only from the verified JWT**, never from the request body or URL. A user can't claim to be someone else.
3. **One choke-point**: all vector searches go through a single function (`app/vectorstore/query.py`), which **refuses to run without a user_id** (it raises `MissingUserScope`) and always adds the filter `user_id = <you>` . Developers can't forget the filter, because there's no other path.
4. All writes and deletes go through `index_store` , keyed by (user, platform, file).
5. Point IDs include the user, so two users indexing the same file get separate copies.
6. **Ownership checks before opening a file**: the file must be in *your* ledger; otherwise you get **404**, not **403**, so the API doesn't even confirm the file exists.
7. **Tests prove it**: two users index identical files, and each sees only their own results.

Why shared collections instead of one database per user? It's the standard multi-tenant pattern for vector databases. It scales to many users without managing thousands of collections, and the indexed

`user_id` filter is cheap. Stricter compliance could use a collection per tenant, at the cost of more operations work.

Layer 3: Protecting third-party credentials (Google/GitHub tokens)

- **Encrypted at rest:** access and refresh tokens are encrypted with **Fernet** (AES-128-CBC + HMAC-SHA256, authenticated encryption) using `TOKEN_ENCRYPTION_KEY` from the environment. A database leak alone does **not** reveal usable tokens.
- **Least privilege:** Drive is **read-only**, and the app verifies the scopes actually granted.
- **OAuth protections:** random, single-use, 10-minute **state** (blocks login-CSRF, where an attacker links their account to yours), and **PKCE** (blocks stolen authorization codes).
- **Revocation:** disconnecting or deleting an account revokes the token **at Google/GitHub**, not just locally.
- Tokens are **never** returned by any API, included in data exports, or written to logs (a log filter redacts tokens, JWTs and OAuth codes).

Layer 4: Protecting the files themselves

- **Original cloud files are not stored.** Drive/GitHub files are downloaded into a **private per-job temp folder**, processed, and deleted immediately. Only extracted text chunks and vectors are kept.
- **Path traversal protection** (`app/core/paths.py`): every user-supplied path is fully resolved (following symlinks, decoding `%2e%2e`, handling drive letters/UNC) and must stay **inside** the allowed base. `../../etc/passwd` tricks fail.
- **Uploads:** only allowed extensions, a size limit, a sanitized filename, and stored per user.
- **Nothing is ever executed or opened on the server.** The original version could open any file via `os.startfile`; that was removed.

Layer 5: Input and output hygiene

- Pydantic validation on every request (types, lengths, enums), with clean 422 errors.
- **Errors never leak internals:** no stack traces, file paths outside the user's scope, URLs or tokens in responses or job error messages.
- The frontend escapes all text (React) and highlights matches with offsets, not raw HTML.

Layer 6: Transport and browser security

- **HTTPS** via Caddy (automatic certificates; HSTS on real domains).
- **Strict Content-Security-Policy:** only our own scripts, styles and fonts; no inline scripts; framing denied.
- Plus `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy` and `Permissions-Policy`.
- CORS is restricted to known origins in development; production is same-origin.

Layer 7: Infrastructure

- **Only Caddy is public** (ports 80/443). Postgres, Qdrant and Redis sit on an **internal Docker network** with no public ports.
- Qdrant requires an **API key**. The database app user is **not a superuser** (least privilege).
- Containers run as a **non-root user**.

- **Secrets live in** `.env` (never in git). The production startup **refuses to run** without strong keys (JWT secret ≥ 32 characters, a valid Fernet encryption key, the Qdrant API key) and without `REDIS_URL`.
- **Encrypted backups:** database dump plus Qdrant snapshots in one archive encrypted with an authenticated cipher; tested backup→restore.
- **Secret scanning:** gitleaks runs on the full git history. (The project history was cleaned once after an old token was found, and that token was revoked.)
- **Observability without leaks:** structured logs with request IDs, with secrets redacted; metrics only on the internal network.

Honest limitations (say these yourself; it builds trust)

1. **Indexed text is not encrypted at rest inside Qdrant/Postgres** (only the tokens are). The mitigation is disk/volume encryption from the host or cloud provider, plus the database being unreachable from outside. Application-level encryption of vectors would break similarity search.
2. **JWT in localStorage** (mitigated by CSP, with a cookie migration planned).
3. **Single worker / single GPU:** fine for personal and small-team use; scaling would mean separate worker processes and a GPU pool.
4. Shared Drives aren't indexed, and there's no query expansion yet ("government documents" $\neq$ "Aadhaar").

Model answer (memorise this shape):

"I secured user data in layers. **Identity:** bcrypt passwords, short-lived signed JWTs with server-side revocation, and rate limits. **Isolation:** every vector and row is tagged with the owner's ID taken only from the verified token, and every search goes through one function that refuses to run without that filter, proven by cross-user tests. **Third-party tokens:** read-only OAuth scopes with PKCE and single-use state, Fernet-encrypted at rest, revoked on disconnect, never logged. **Files:** cloud originals aren't stored, paths are traversal-safe, and nothing is executed server-side. **Infrastructure:** HTTPS with a strict CSP, databases on a private network behind keys, non-root containers, secrets outside git, encrypted backups. The main remaining gap is encryption at rest for the indexed text, which I'd handle at the disk layer, and moving the session to httpOnly cookies."

Part 10: The database (what's stored where)

PostgreSQL tables: | Table | Purpose | |---|---| | `users` | id (UUID), email, name, bcrypt hash, token_version, onboarding_completed | | `platform_connections` | per user and platform: account name/email, **encrypted** access/refresh tokens and token JSON (which includes the granted scopes), connected flag | | `oauth_states` | random state, user, platform, PKCE verifier, expiry, used flag | | `local_storage_folders` | registered folders per user (unique per user+path) | | `indexing_jobs` | the job queue: status, priority, counters, errors, stage timings | | `indexing_job_errors` | per-file errors (sanitized, capped at 200 per job) | | `indexed_files` | **the ledger:** one row per file (user, platform, source_id, version, status, type, size, modified, link) |

Qdrant collections (prefix `cogniseek_v2_`): `text` (384-d MiniLM), `image` (512-d CLIP), `audio` (384-d), `video` (transcripts, 384-d), and `video_frames` (512-d CLIP). Each point's payload holds `user_id`, `platform`, `source_id`, file name, display path, type, version, chunk index, the chunk text, and frame number/timestamp. Keyword indexes on `user_id`, `platform`, `source_id` and type make filtering fast.

Why both Postgres and Qdrant? Qdrant is excellent at "find nearest vectors with a filter". Per-file facts (version, status, errors, counts, ownership) are relational. Keeping them in Postgres makes change detection, the dashboard and ownership checks simple, indexed SQL queries. (The original version kept the same data in pickle files *and* Qdrant, and the two drifted apart, which caused ghost results.)

Migrations: Alembic, with 15 versioned migrations. Schema changes are code-reviewed, reversible and applied with `alembic upgrade head`.

Part 11: The frontend

- **React 19 + TypeScript + Vite + Tailwind**, with animations (motion) and dark mode.
- **Pages:** Login/Register → **Onboarding** (connect platforms → choose a priority platform → start) → **Dashboard** (hero search, filter chips, stats, Recently indexed, Recent searches, results + possible matches) → **Platforms** (connect/disconnect/re-index, local folders) → **Indexing Center** (live job progress, errors, cancel, "Index next", run history, "Where the time went") → **Account settings** (export, change password, delete account).
- **Data layer: TanStack Query** handles caching per user, one shared poller for jobs (every 2 s while active, off when idle), and invalidation after actions.
- **Typed API client** generated from the backend's OpenAPI schema.
- **Race safety:** each new search **aborts** the previous request, and each search has its own cache entry, so a slow old response can never overwrite newer results.
- **Accessibility:** keyboard navigation, an autocomplete combobox, focus-trapped dialogs, AA colour contrast (checked automatically with axe), and reduced-motion support.
- **Priority onboarding idea:** indexing everything takes time, so the user picks the platform they care about most. It's indexed first, search works immediately, and the rest index in the background with a live banner.

Part 12: Testing and quality

Level	Tool	What it covers
Backend unit/integration	pytest (450+ tests)	auth, isolation, path safety, OAuth state/PKCE, encryption, job queue, connectors (mocked HTTP), ranking, the indexing pipeline
Frontend	Vitest + Testing Library + MSW (70+ tests)	user flows against a fake backend with the real types: search races, filters, errors, onboarding, downloads
End-to-end	Playwright	real browser + real backend: register → onboarding → index → search → download → delete account, with axe accessibility checks
Search quality	Eval harness	Precision@5, MRR, false positives across about 80 labelled queries
Performance	Benchmark script	per-stage timings, throughput, before/after
Security	gitleaks, pip-audit, npm audit	secrets and vulnerable dependencies

Tests are **hermetic**: each run gets its own temporary database and in-memory Qdrant, with fake AI models (deterministic vectors), so they're fast and don't touch real data.

Part 13: Deployment

```
git clone ... && cp .env.example .env && python scripts/generate_secrets.py && docker compose up -d --build
```

- **Services:** caddy (public), backend, postgres, qdrant, redis (internal).
- **Backend image:** multi-stage, non-root, CPU by default (an optional GPU variant); the model cache lives in a volume (downloaded once); it runs migrations then uvicorn with 1 worker (because the indexing worker and GPU models live in-process).
- **Health:** `/health` (is the process alive?) vs `/ready` (are the DB, Qdrant, Redis and models reachable?). Docker starts services in dependency order using these.
- **Verified from a fresh clone:** build + model download about 13.5 minutes the first time; the full flow worked; backup/restore verified.

Part 14: The journey (great for "tell me about the project" and behavioural questions)

The project started as a working prototype with serious problems, found in a full QA audit:

- **Critical security holes:** unauthenticated endpoints could delete or open any file on the server; every user saw everyone's files; a hard-coded JWT secret; tokens in plain text; a leaked token in git history.
- **Reliability:** one error could freeze indexing forever; two indexes (pickle + Qdrant) drifted apart, so deleted files kept appearing; video search scores were always zero.
- **Frontend:** fake data on screen, results hidden by client-side filters, broken endpoints.

How it was fixed (in phases, each reviewed and tested before merging):

0. Baseline: clean git history (the leaked token removed and revoked), config in `.env`, Docker, a test harness.
1. Security: auth everywhere, path safety, OAuth state, token encryption, CORS/headers.
2. Indexing reliability: a Postgres job queue, per-file error isolation, restart recovery.
3. One index: Qdrant only, with per-user isolation and a Postgres ledger (pickles removed).
4. Search quality: hybrid retrieval, calibration, an evaluation harness; nonsense queries went from 61 junk results to about 0.
5. Connectors: Drive web OAuth + PKCE, the GitHub Trees API, change-detection-before-download, visual search fixes.
6. Frontend: rebuilt on real data with tests; the original design restored; priority onboarding. 7A. Security hardening + Docker/HTTPS deployment. 7A.5: indexing performance ($3.7\times$ faster), then a Drive thread-safety fix. 7B. CI (GitHub Actions), documentation, final QA and the v1.0.0 release.

Real bug stories (STAR format: Situation, Task, Action, Result):

1. **PKCE bug:** Drive sign-in failed with "Missing code verifier". *Cause:* the library only generates the verifier inside `authorization_url()`, but we saved it to the DB before calling that, so we saved nothing. *Fix:* generate the verifier explicitly up front; a regression test uses the **real** Google library (the earlier tests used a fake, which is why they missed it). *Lesson:* mocks can hide integration bugs.
2. **Granted-scope bug:** Drive connected but indexing got 403. *Cause:* Google's consent screen has per-permission checkboxes; Drive wasn't ticked, yet we stored the scopes we *requested*, not those *granted*. *Fix:* verify the granted scopes at callback and fail clearly.

3. **"Dog" returned nothing:** image search secretly worked only through filenames. The test set had the same blind spot (every image query shared a word with its filename). *Fix:* added visual-only test queries, then recalibrated the CLIP threshold from data. P@5 went from 0.06 to 0.72. *Lesson:* your evaluation set can overfit too.
 4. **Flaky tests:** tests failed randomly. *Cause:* every run dropped and recreated one shared test database, so a second process (IDE test discovery) broke a run in progress. *Fix:* a unique database per run. Frontend flakiness came from animations and real timers; fixed with deterministic timing.
 5. **Qdrant request-size bug:** huge generated log files made one upsert exceed Qdrant's request size limit. *Fix:* batched writes, size limits, junk-file exclusion.
 6. **Performance:** profiling showed OCR on the CPU while the GPU sat idle 86% of the time; fixing the top bottlenecks gave 3.7× faster indexing with quality verified unchanged.
 7. **Drive thread-safety bug:** after the parallel pipeline shipped, Drive indexing failed on random files with SSL errors (`DECRYPTION_FAILED_OR_BAD_RECORD_MAC`). *Cause:* four download threads shared one `httplib2.Http` connection, which is not thread-safe; tests used a fake Drive client and the benchmark used local files, so neither exercised it. A read-only dry-run on the old code crashed the process (access violation in `ssl.recv_into`, 3 of 3 runs). *Fix:* one authorized transport per thread (and one `requests.Session` per thread for GitHub), TLS errors retried on a fresh connection, and tests with a real local HTTPS server. *Result:* 68/68 Drive files downloaded with 4 and 8 workers, 0 errors. *Lesson:* concurrency bugs hide behind fakes; test the real transport.
-

Part 15: Likely interview questions (quick answers)

1. **Why a vector database instead of SQL LIKE search?** LIKE only matches exact words. Vectors capture meaning, so "car" finds "automobile", and images can be found by content.
 2. **How do you avoid irrelevant results?** Calibrated thresholds (margins against neutral prompts, a per-video baseline), evidence requirements, a separate low-confidence tier, and an evaluation harness with negative queries.
 3. **How would you scale to 10,000 users?** Move the indexing worker to separate processes with a GPU pool, which the queue already supports via SKIP LOCKED; scale the stateless API horizontally behind Caddy; run Qdrant in a cluster with sharding by `user_id`; Redis is already shared; Postgres gets read replicas.
 4. **What happens if indexing crashes midway?** Per-file isolation; on restart, running jobs become "failed: interrupted"; the next run resumes cheaply because unchanged files are skipped by version.
 5. **Why both Postgres and Qdrant?** Each is used for what it's best at; the old dual-pickle design drifted, so now there's one owner per kind of data.
 6. **How do you know search is good?** Labelled queries with P@5/MRR, negatives, and no-regression rules on every change.
 7. **Biggest challenge?** Calibrating CLIP: raw similarity can't be thresholded, so scores had to be measured relative to neutral prompts and, for videos, to each video's own baseline.
 8. **What would you do next?** `httpOnly`-cookie auth, query expansion and synonyms, Shared Drives, more connectors (Notion, Slack, OneDrive), disk encryption, and separate worker processes.
 9. **How is a user's data deleted?** Delete account revokes the provider tokens, removes all vectors by `user_id`, all rows and uploads, and invalidates the session immediately.
 10. **Why one indexing worker?** The GPU models are shared; parallel jobs would fight for VRAM. The A/B test chose per-model locks with 4 files in flight as the fastest safe option.
-

Tip: before any interview, reread Parts 1, 9 and 14 and the numbers in Parts 6.6 and 8.6. Those three cover 80% of what people ask.